



Birzeit University
Faculty of Engineering and Technology
Department of Electrical and Computer Engineering
ENCS5342 – Information Retrieval with Applications of NLP (Term 1252)
Final Course Project

1. Overview

The final project is a significant component of ENCS5342 and gives students an opportunity to apply the concepts, methods, and tools covered throughout the course to a concrete NLP problem. Projects are completed in **teams of 2–3 students**. Every team must select **one of the three tracks** described in this document and register their choice by the deadline specified in Section 2.

All tracks require the submission of a Jupyter Notebook file as the primary deliverable. The notebook must follow the structure defined in Section 5 and must be well-organised, clearly written, and demonstrate both technical depth and conceptual understanding.

All teams — regardless of track — must create a **private GitHub repository for their project** and share the repository link with the instructor. See Section 6 for details.

2. Key Dates

Milestone	Date	Notes
Team & track registration	9 June 2026	All teams must register by this date. See Section 2.1 for the registration link.
Track 2 & 3 approval deadline	11 June 2026	Teams on Tracks 2 and 3 must have received instructor approval of their topic/paper by this date.
Track 2 presentations	22–25 June 2026	Presentation slots (date and time) to be selected by students via a shared Excel sheet (link to be shared by instructor). Students must be ready to present by 22 June.
Final notebook submission — all tracks	25 June 2026	Last teaching day. Submit the final Jupyter Notebook (and all required files) via Moodle/ITC by 11:59 PM.
Track 1 & 3 in-person discussion meetings	27 June – 12 July 2026	Scheduled during the final exam period. Discussion slots to be selected by students via a shared Excel sheet (link to be shared by instructor).

Late submissions will not be accepted unless prior written approval has been granted by the instructor.

2.1 Team & Track Registration

Register your team and track selection using the link below. Provide the full names and student IDs of all team members, the selected track, and — for Track 2 — the chosen paper title and DOI/URL, or — for Track 3 — the one-paragraph project proposal.

Team & Track Registration Form:

https://docs.google.com/spreadsheets/d/1t8_8QTKUqv75EKwyKv1nLtlx3ydBgAt5

Track 2 and Track 3 topics require instructor approval. Approval requests must be submitted no later than Thursday, 11 June 2026. Approval will be granted via Ritaj, email, or by indicating the decision directly on the registration form.

3. Project Tracks

Teams must select exactly one of the three tracks below. Switching tracks after the registration deadline requires written approval from the instructor.

Track 1 – Default Project: Part-of-Speech (POS) Tagging

Description

Part-of-Speech (POS) tagging is one of the most fundamental tasks in Natural Language Processing. It is the process of assigning a grammatical category — such as noun, verb, adjective, or preposition — to each word in a sentence, taking into account both the word's definition and its context within the sentence. Although it may appear straightforward, POS tagging is a non-trivial disambiguation problem: the same word form can belong to different categories depending on context (e.g., "book a flight" vs. "read a book"), and real text contains rare words, named entities, and domain-specific vocabulary that make the task genuinely challenging.

POS tagging sits at the foundation of the NLP pipeline and is a prerequisite for many higher-level tasks. Accurate POS tags improve the performance of dependency parsers, named entity recognizers, semantic role labelers, machine translation systems, and information extraction pipelines. In search engines, POS information helps disambiguate queries; in speech synthesis, it guides correct pronunciation of homographs; in grammar checking tools, it enables detection of agreement errors. Understanding how to build and evaluate a tagger is therefore a core NLP competency.

This task is chosen as the default project for ENC5342 for several reasons. First, it is a well-defined sequence labelling problem with a clear, quantitative evaluation metric (tagging accuracy), which allows teams to measure progress objectively and compare approaches fairly. Second, it naturally exercises the full range of models covered in the course — from simple probabilistic baselines to RNNs, LSTMs, and Transformer encoders — making it an ideal sandbox for hands-on experimentation. Third, the English Penn Treebank is a gold-standard, widely-cited benchmark, so students can contextualize their results against a rich body of published literature. Finally, the task is tractable enough that meaningful results can be obtained within the project timeline, yet rich enough to reward deeper investigation.

Teams will build and evaluate a POS tagger for English entirely from scratch — from data loading and preprocessing through model implementation, training, and evaluation. This end-to-end ownership is intentional: every step of the pipeline, including parsing the CoNLL format, building a vocabulary, handling unknown words, padding and batching variable-length sequences, and computing evaluation metrics, is a core NLP engineering skill that you are expected to develop through this project.

Note: POS tagging is a foundational and important NLP task, but it is also a relatively well-studied problem with known solutions. Track 1 is designed to give all students a solid, manageable starting point. Students who are looking for a greater intellectual challenge, wish to tackle an open research problem, or want to work on a task with broader real-world impact are strongly encouraged to consider Track 3 instead.

Dataset

The project uses the English Penn Treebank corpus in the basic Stanford-style dependency format — one of the most widely used benchmarks in NLP research. The data is distributed in the CoNLL column-based format: each sentence is a block of tab-separated lines (one line per token) followed by a blank line.

Two data files will be shared with students via the Google Drive folder below. A third file (the test set) is held out by the instructor and will not be released to students at any point during the project.

Dataset Folder (Google Drive):

<https://drive.google.com/drive/folders/1vcmTMEQuETc7wCov01Z39rEyK04GSq7r>

File	Size	Purpose
en-universal-train.conll	~37,840 sentences	Training set. Use this to train all models.
en-universal-dev.conll	~1,992 sentences	Validation set. Use this for hyperparameter tuning and all intermediate evaluation. Report all results on this set.
Test set (withheld)	—	Held out by the instructor. Evaluated after submission.

The dataset is copyrighted material and must not be distributed or shared outside of this course.

Col	Field	Description
1	Word ID	Integer token index, starting at 1 for each new sentence.
2	Word Form	The surface form of the token as it appears in the text.
3	Lemma	Base/dictionary form. Not used in this project.
4	Coarse-grained POS	Universal POS tag (e.g., NOUN, VERB, DET, ADP).
5	Fine-grained POS	Penn Treebank POS tag (e.g., NN, VBD, DT, IN).
6	Features	Morphological features. Not used in this project.
7	Head ID	ID of the syntactic head/parent. Not used in this project.
8	Dependency Relation	Grammatical relation to head. Not used in this project.
9–10	Projective fields	Not used in this project.

Example sentence from the corpus:

ID	Form	Lemma	CPOS	FPOS	Feats	Head	Rel	PH	PR
1	Or	—	CONJ	CC	—	2	cc	—	—
2	was	—	VERB	VBD	—	0	ROOT	—	—
3	it	—	PRON	PRP	—	2	nsubj	—	—
4	because	—	ADP	IN	—	8	mark	—	—
5	Ms.	—	NOUN	NNP	—	6	compmod	—	—
6	Collins	—	NOUN	NNP	—	8	nsubj	—	—
7	had	—	VERB	VBD	—	8	aux	—	—
8	gone	—	VERB	VRN	—	2	dep	—	—
9	?	—	.	.	—	2	p	—	—

POS Tag Target: Teams should predict a combined joint tag merging the coarse- and fine-grained tags — e.g., NOUN-NNP, VERB-VBD, PRON-PRP, ADP-IN.

Scope & Expectations

Teams are free to employ any techniques or architectures, including but not limited to:

- Rule-based or lexicon-based baselines
- Probabilistic models (e.g., HMM, maximum-entropy taggers)
- Machine-learning classifiers (e.g., logistic regression, SVMs with hand-crafted features)
- Recurrent neural networks (simple RNN, LSTM, GRU — bidirectional variants encouraged)
- Transformer-based encoders (self-attention implemented from scratch or adapted from a minimal reference)

Teams may use any programming language or framework. We expect the large majority of teams to use Python with PyTorch.

Teams must implement and compare at least two distinct approaches (e.g., a simple baseline and a neural model, or two different neural architectures), reporting training set and development set results for each.

Important note on originality: publicly available code for POS tagging exists online. Reproducing another team's or another author's implementation without meaningful modification, understanding, and extension will score very low on the Code & Techniques component, regardless of the accuracy achieved. The value-add your team brings — in design decisions, experiments, analysis, and insight — is what is being graded.

To achieve stronger performance, teams are encouraged to apply sound ML engineering practices. The more relevant techniques are applied thoughtfully, the better the score within the code quality component. Relevant techniques include:

- Regularization (L1/L2 weight decay)
- Dropout (standard, variational, or embedding dropout)
- Gradient clipping to prevent exploding gradients
- Early stopping based on development set performance
- Learning rate scheduling (e.g., warm-up, cosine annealing)
- Batch normalization or layer normalization
- Cross-validation or multiple random seeds for robust evaluation
- Hyperparameter search (grid search, random search, or Bayesian optimization)
- Pre-trained word embeddings as input features (e.g., word2vec, GloVe, fastText)

Required Evaluation Metrics

POS tagging is a multi-class classification task, and overall accuracy alone is insufficient — it can mask poor performance on minority tag classes. For every model and every data split reported, teams must include all of the following metrics:

- Per-class precision, recall, and F1-score — reported for every POS tag in the tag set.
- Micro-averaged precision, recall, and F1-score.
- Macro-averaged precision, recall, and F1-score.
- Overall token-level accuracy — the percentage of tokens assigned the correct joint tag.

The distinction between micro- and macro-averaged scores is particularly important here. The Penn Treebank tag distribution is highly skewed: frequent tags such as NOUN-NN and punctuation dominate the corpus, while rare tags such as existential-there (EX), foreign words (FW), or list items (LS) appear very infrequently. A model that sacrifices these rare classes entirely can still achieve high micro-averaged scores. Macro-averaged F1 will expose this. Teams are expected to analyze and discuss any significant performance gap between frequent and rare tag classes in their error analysis.

These metrics must be reported using a clearly formatted table or heatmap in the Results section of the notebook. Students are expected to understand what each number means, not just print the output.

This is a course project whose primary purpose is learning. Teams are strongly expected to implement their core models from scratch as much as possible, rather than calling high-level pre-built pipelines. Loading a fully pre-trained model via a high-level API with no further implementation effort is the minimum-effort approach and will receive minimal credit in the from-scratch component, regardless of accuracy. The grading rubric explicitly rewards the effort to understand and build.

Grading Breakdown – Track 1

Performance (20% total)

Sub-component	Weight	Details
Training set performance	5%	Per-class and averaged precision, recall, and F1-score on the training set, reported in the notebook. Demonstrates the model is learning from data.
Validation (development) set performance	5%	Full metrics on the development set for all compared models. All intermediate experiments must be reported on this set.
Test set accuracy	10%	Final token-level accuracy on the held-out test set, evaluated by the instructor after submission.

Code & Techniques — Jupyter Notebook (40% total)

Sub-component	Weight	Details
Coding from scratch	15%	Degree to which core model components (architecture, forward pass, training loop) are implemented by the team rather than delegated to high-level pre-built pipelines. Partial credit awarded proportionally to depth of implementation.
Overall code quality & ML techniques	25%	Clarity, organization, and reproducibility of the notebook; appropriateness and depth of methods; use of sound ML engineering practices (regularization, scheduling, early stopping, etc.); ablation experiments.

Discussion (40%)

Component	Weight	Details
In-Person Discussion	40%	Assessed during the scheduled in-person meeting. Grade is based on the student's ability to: (1) correctly answer questions from the instructor about their design choices, implementation, results, and analysis; and (2) re-implement or explain specific parts of their code on the spot if requested.

Track 2 – Paper Presentation

Description

Teams select a published NLP research paper and produce a thorough, pedagogically valuable Jupyter Notebook presentation of it. The goal is not merely to summarize the paper, but to explain, contextualize, reproduce, and critically evaluate it for an audience of fellow NLP students.

Paper Selection & Approval

- The paper must be relevant to the topics covered in ENCS5342.
- The paper must have been published within the last five years (2021–2026), OR it may be an older landmark paper of well-established significance (subject to instructor approval).
- Each team must select a different paper — first-come, first-served on approval.
- Paper approval must be obtained from the instructor no later than Thursday, 11 June 2026.
- The instructor can assist teams in locating and downloading any paper they wish to access. Do not hesitate to ask.

Suggested Topic Areas (LLMs & Modern NLP)

The following are recommended topic areas for teams who would like guidance. Papers within these areas are plentiful and highly relevant to current NLP research:

- LLM Agents — autonomous agents built on top of large language models (tool use, planning, multi-agent frameworks).
- Sycophancy — the tendency of LLMs to agree with users regardless of correctness; detection and mitigation methods.
- Hallucination & Factuality — understanding, measuring, and reducing factual errors in LLM outputs.
- Reasoning in LLMs — mathematical and logical reasoning, chain-of-thought prompting, scratchpad methods.
- Model Pruning — structured and unstructured pruning techniques for compressing large language models.
- Model Quantization — reducing LLM memory and compute requirements through quantization (e.g., GPTQ, QLoRA, AWQ).
- Retrieval-Augmented Generation (RAG) — combining retrieval systems with generation models to improve factuality.
- Instruction Tuning & RLHF — aligning language models to follow instructions using human feedback.
- Efficient Fine-Tuning — parameter-efficient methods such as LoRA, adapter tuning, and prefix tuning.

Teams are not restricted to this list. Any relevant, high-quality paper is welcome subject to instructor approval.

Deliverables

Teams must prepare a Jupyter Notebook that serves as a self-contained presentation. It should include:

1. A structured introduction to the problem the paper addresses and its significance.
2. An explanation of the proposed method, including relevant background and mathematical formulations.
3. Figures, diagrams, or visualizations that aid understanding (original or reproduced from the paper with clear attribution).
4. A re-implementation or demonstration of key components where feasible, or a thoroughly explained code walkthrough.
5. A critical discussion: strengths and limitations; relation to other work; open questions.

6. A live presentation to the class.

Note on the presentation schedule: students must be fully ready to present by 22 June 2026. The final Jupyter Notebook file may be submitted up to 25 June 2026 (11:59 PM).

Grading Breakdown – Track 2

Component	Weight	Details
Jupyter Notebook	40%	Completeness, clarity, and depth of explanation; quality of visualizations and code demonstrations; overall organization and notebook structure (see Section 5).
Presentation	40%	Clarity and quality of the live class presentation; ability to explain complex ideas accessibly; time management; use of visual aids.
Discussion (Q&A)	20%	Ability to answer questions from the instructor and peers; depth of understanding of the paper and its broader context.

Track 3 – Custom Course Project

Description

Teams define and solve their own NLP problem. This is the most open-ended and ambitious track, and it rewards creativity, initiative, and rigour. The chosen problem must be genuinely related to NLP and must not have been completed (in any form) for any other course or project prior to ENC5342. Work produced here may, however, be extended and used later in a graduation project.

Why Choose the Custom Course Project?

Consider this track if you:

- Have a research problem or idea you are excited about and possibly already thinking about.
- Want to try something different and original — defining your own goal rather than following a prescribed path.
- Want to experience the full research process: formulating a research question, finding data and tools, designing an approach, running experiments, and evaluating results.
- Want a project you could later build on for your graduation project.

Important constraint: The final project for ENC5342 must substantively involve both human language and neural networks — the core topics of this course. Projects that do not meaningfully engage with both will not be approved.

Advanced Topics & Bonus Credit

While the default project (Track 1) focuses on POS tagging — a foundational but relatively simple sequence labelling task — the landscape of NLP offers a rich range of more advanced and impactful problems that are well within reach for a motivated team. Students who choose Track 3 are encouraged to work on downstream NLP tasks that build on lower-level linguistic representations and tackle problems closer to real language understanding and real-world applications. Examples of such tasks include:

- **Clinical NLP** — fine-tune transformers for disease/medication extraction, ICD code prediction, or mental health signal detection from social media.
- **Misinformation & Fake News Detection** — detect false claims, propaganda, or AI-generated content using neural models that capture reasoning beyond surface features.
- **Hate Speech Detection** — identify offensive or abusive language online, with focus on implicit hate, cross-lingual settings, and demographic fairness.

- **Argument Mining** — detect claims, premises, and support/attack relations in text; applications in legal reasoning and automated essay scoring.
- **LLM Hallucination Detection** — identify factually incorrect or unsupported claims in LLM-generated text using classifiers or retrieval-augmented approaches.
- **Legal NLP** — court judgment prediction, clause classification, contract review, or statute retrieval using neural models trained on legal text.
- **Low-Resource & Cross-Lingual NLP** — fine-tune multilingual models (mBERT, XLM-R) for low-resource languages using zero-shot or few-shot transfer.
- **Automated Essay Scoring** — neural models for automated grading and feedback generation on student-written essays or short answers.
- **Dialectal & Offensive Arabic NLP** — detect hate speech or process dialectal Arabic (Palestinian, Levantine, Egyptian) where standard MSA models fail.

Teams working on advanced Track 3 projects that go substantially beyond the scope of Track 1 in ambition, technical depth, and analytical rigour are eligible for bonus credit of 10% added to their final course grade, as well as an extended submission deadline, both awarded at the instructor's discretion.

Topic Approval

Teams must submit a one-paragraph project proposal to the instructor for approval no later than Thursday, 11 June 2026. The proposal should describe:

- The NLP problem being addressed and its significance.
- The proposed approach or methodology at a high level.
- The dataset(s) to be used.
- The expected outcome and how results will be evaluated.

Projects that are insufficiently related to NLP, do not involve neural networks, or reproduce published work without meaningful extension will not be approved.

Important note on originality: adapting or reproducing an existing publicly available implementation without meaningful modification, understanding, and extension will score very low on the Code & Techniques component. Teams are expected to go beyond what any existing code provides — through novel design choices, additional experiments, thorough analysis, or meaningful extensions.

Grading Breakdown – Track 3

Component	Weight	Details
Performance	10%	Evaluated based on the appropriateness and rigour of the metrics chosen for the task. Results must be reported on both the training and validation sets, with a clear and honest interpretation of the numbers.
Project Importance & Relevance	10%	Significance and novelty of the chosen problem; clarity of the research question and motivation; quality of the approved proposal.
Code & Techniques (Jupyter Notebook)	40%	Quality, reproducibility, and depth of implementation; appropriateness of methods; use of sound ML engineering practices; experimental design.
In-Person Discussion	40%	Assessed during the scheduled in-person meeting. Grade is based on the student's ability to: (1) correctly answer questions from the instructor about their design choices, implementation, results, and analysis; and (2) re-implement or explain specific parts of their code on the spot if requested.

4. Submission Requirements

Final Submission Package

Submit a single ZIP file named `ENC5342_Project_<TrackNumber>_<IDs>.zip` to Moodle/ITC by 11:59 PM on 25 June 2026. The ZIP must contain:

1. The Jupyter Notebook (.ipynb), fully executed with all outputs visible.
2. A PDF export of the notebook.
3. Any supplementary scripts required to reproduce results (do not include large raw datasets; provide download/access instructions instead).
4. A README.txt describing how to run the code and listing all library dependencies with versions.

Reproducibility

All results reported in the notebook must be reproducible by running the notebook from start to finish. Set random seeds explicitly wherever randomness is involved. If external resources (pre-trained embeddings, APIs) are used, document them clearly with download links or instructions.

5. Required Notebook Structure

The Jupyter Notebook for all tracks must be organised into the following sections, in order. Each section should contain both explanatory text (Markdown cells) and, where applicable, code cells.

#	Section	What to Include
1	Abstract	A concise summary of the problem, approach, and key findings.
2	Introduction	Motivation for the problem; research questions or goals; overview of the approach; summary of contributions.
3	Prior & Related Work	Review of relevant prior work. Cite and briefly discuss at least 1 related paper. Explain how your work relates to or differs from them.
4	Model	Detailed description of the model(s) or method(s) used. Include architectural diagrams, equations, and design decisions.
5	Data	Description of the dataset(s): source, size, format, preprocessing steps, and any data analysis (e.g., tag distribution, sentence length statistics).
6	Experiments	Experimental setup: hyperparameters, training procedure, baseline(s) used for comparison. Clearly state all evaluation metrics used: per-class precision/recall/F1, micro- and macro-averaged scores, etc.
7	Results	Quantitative results tables covering training and validation performance for all compared models. Must include per-class and averaged metrics. Confusion matrices and learning curves are strongly encouraged.
8	Analysis & Conclusion	Error analysis including discussion of per-class performance gaps; discussion of findings; limitations; overall conclusion.
9	AI Usage Statement	Disclosure of any AI tools used during the project (see Section 7). Mandatory for all teams.
10	References	Full bibliography for all papers, datasets, libraries, and code resources cited in the notebook.

Notebooks that do not follow this structure will be penalized in the code/notebook grade.

6. GitHub Repository Requirement

All teams must maintain a private GitHub repository for their project. This serves two purposes: (1) it gives the instructor visibility into team progress without requiring formal check-ins, and (2) it teaches professional version control practices that are essential in both research and industry.

Requirements

- Create a private GitHub repository as soon as your team is formed (by the registration deadline, 9 June 2026).
- Add the instructor as a collaborator so progress can be monitored.
- Share the repository link with the instructor no later than the registration deadline.
- Commit and push your work regularly throughout the project period — do not make one large commit at the end.

Important: Commit history will be reviewed. A repository with a single large commit the night before the deadline is a red flag and may result in a deduction from the code quality component. Sustained, incremental commits demonstrate genuine ongoing effort.

Repository Contents

At a minimum, the repository should contain:

- The Jupyter Notebook (updated regularly as work progresses).
- A README.md with the project title, team members, track, and a brief description.
- Any scripts or utilities used to preprocess data or run experiments.

7. Ethical Use of AI Tools

AI-assisted tools (e.g., GitHub Copilot, ChatGPT, Claude, Gemini, and similar) are permitted in this course project subject to the following conditions. Misuse is treated as academic dishonesty.

Permitted Uses

- Debugging code, understanding error messages, or looking up syntax.
- Getting feedback on writing clarity or grammar.
- Exploring ideas or understanding concepts (treat this like consulting a tutor).
- Generating boilerplate or utility code (e.g., data loaders, plotting helpers).

Not Permitted

- Having an AI tool write the core modelling, experimental, or analytical sections of the notebook on your behalf.
- Submitting AI-generated text as your own analysis, discussion, or conclusions without meaningful independent editing and verification.
- Using AI tools to generate insights you have not independently understood.

Mandatory AI Usage Statement

Every team must include an AI Usage Statement as the final section of their Jupyter Notebook (Section 9 per the structure in Section 5). This statement must describe:

1. Which AI tools were used (if any).
2. Specifically, how they were used (e.g., 'We used GitHub Copilot to autocomplete boilerplate data-loading code and ChatGPT to proofread the Introduction section.').
3. Which parts of the notebook were written or created entirely by the team without AI assistance.

Teams that did not use any AI tools must state this explicitly. Omission of this statement when AI tools were used is treated as a violation of academic integrity.

Guiding Principle

You are responsible for every claim, result, and line of code in your submission. The discussion and in-person Q&A components are specifically designed to assess genuine understanding — be prepared to explain and defend every aspect of your project.

8. Collaboration Policy & Academic Integrity

Collaboration Policy

Projects are team efforts. Within your team, full collaboration is expected and encouraged. Across teams, the following applies:

- It is acceptable to use existing code, libraries, or resources found online, but you must document every such source clearly in the notebook and in your references.
- You will be graded on your value-add — what your team understood, built, extended, or analyzed beyond what any external resource provided.
- Each team member must include a brief individual contribution statement in the notebook (can be placed in the README or as a note in the Abstract section), describing their specific role and contributions.
- In almost all cases every team member receives the same project score. However, the instructor reserves the right to differentiate scores in egregious cases where contribution is clearly imbalanced.

Academic Integrity

The following are violations of academic integrity and will result in a grade of zero for the project, with the possibility of further disciplinary action per university policy:

- Plagiarism of text, code, or results from any source — published papers, online repositories, other students' work, or AI-generated content submitted without attribution.
- Sharing project materials with other teams or using another team's materials.
- Re-submitting work done (in any form) for another course or prior project. Track 3 projects must be new work done specifically for ENCS5342.
- Fabricating or manipulating experimental results or reported metrics.

Proper attribution is mandatory. When using third-party code, datasets, embeddings, or any external resource, cite the source clearly in the notebook.

9. Grading Summary

Component	Track 1	Track 2	Track 3
Performance – Training set (precision/recall/F1/accuracy)	5%	—	5%
Performance – Validation set (precision/recall/F1/accuracy)	5%	—	5%
Performance – Test set accuracy (evaluated by instructor)	10%	—	—
Project Importance & Relevance & Performance	—	—	10%
Code – Coding from scratch	15%	—	—
Code – Overall quality & ML techniques	25%	40%	40%
Presentation	—	40%	—

Component	Track 1	Track 2	Track 3
Discussion / Q&A	40%	20%	40%
Total	100%	100%	100%

10. Tips for Success

- Start early. Data exploration, model training, and writing all take far more time than expected.
- Create your GitHub repository on day one. Commit something — even just the README — to get into the habit.
- Divide work fairly within your team and agree on a workflow (branch strategy, commit conventions) from the start.
- Document your experiments as you go. Do not leave the analysis and discussion sections until the last week.
- For Track 1: the from-scratch coding component rewards genuine implementation effort. A hand-built model you truly understand is worth more than a pipeline you cannot explain.
- For Track 1: apply at least 2–3 ML engineering techniques (dropout, early stopping, LR scheduling, etc.) and report their effect in your ablation experiments.
- For Track 2: choose a paper you are genuinely curious about. Enthusiasm and deep understanding show clearly in the presentation and Q&A.
- For Track 3: define a focused, achievable scope early. A well-executed small project is significantly better than an overambitious incomplete one.
- Visit the instructor/TAs during office hours if you are uncertain about your approach, paper choice, or project proposal.
- Read the grading rubrics carefully before you start — they tell you exactly where every percentage point comes from.

Good luck — and have fun!